SAVITRIBAI PHULE PUNE UNIVERSITY

B.E. Project Report On

"Titanic Survival Explorer Using Machine Learning"

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENT
FOR THE B.E. COMPUTER ENGINEERING

BACHELOR OF ENGINEERING

(Computer Engineering (SEM-VII))

SUBMITTED BY

Sarang Deshpande - 4519
Sai Abhang - 4501
Akshad Ambekar - 4504
Aditya Akolkar - 4503

UNDER THE GUIDANCE OF

Dr. G. D. Puri

DURING THE ACADEMIC YEAR 2026–2027



DEPARTMENT OF COMPUTER ENGINEERING
AMRUTVAHINI COLLEGE OF ENGINEERING, SANGAMNER-422608

ACADEMIC YEAR 2026–2027

AMRUTVAHINI COLLEGE OF ENGINEERING

SANGAMNER

Department of Computer Engineering

Certificate

This is to certify that the project entitled **"Titanic Survival Explorer Using Machine Learning"** has been carried out satisfactorily by **Sarang Deshpande** (Roll No. 4519), **Sai Abhang** (Roll No. 4501), **Akshad Ambekar** (Roll No. 4504) & **Aditya Akolkar** (Roll No. 4503) in partial fulfillment of the requirements for the Degree of fourth year of engineering in computer engineering of Savitribai Phule Pune University, during the academic year 2026–2027.

Dr. G. D. Puri
Project Guide

Dr. S. K. Sonkar
Head of Department

Contents

Abstract **2**
1 Introduction **3**
1.1 Background ... 3
1.2 Problem Statement ... 3
1.3 Objectives ... 3
1.4 Scope of the Project ... 3
2 Technologies Used **4**
2.1 Frontend Technologies ... 4
2.1.1 Streamlit ... 4
2.1.2 Streamlit Layout and Input Widgets ... 4
2.1.3 Caching with st.cache_resource ... 4
2.1.4 File Upload and Download ... 4
2.2 Data and Machine-Learning Technologies ... 4
2.2.1 Python ... 4
2.2.2 pandas ... 4
2.2.3 NumPy ... 5
2.2.4 scikit-learn ... 5
2.2.5 Random Forest Classifier ... 5
2.3 Dataset ... 5
2.3.1 Kaggle Titanic Dataset ... 5
2.4 Infrastructure ... 5
2.4.1 Runtime Environment ... 5
3 System Architecture **6**
3.1 Overview ... 6
3.2 Application Module Architecture ... 6
3.3 Model Training and Evaluation Architecture ... 6
3.4 Feature and Data Schema Design ... 7
3.5 Prediction Flow ... 7
4 Implementation Details **9**
4.1 Project Structure ... 9
4.2 Data Handling Implementation ... 9
4.2.1 Configuration and Constants ... 9
4.2.2 Reading CSV Files ... 9
4.2.3 Feature Engineering ... 9
4.3 Model Implementation ... 10
4.3.1 Preprocessing and Classifier Pipeline ... 10
4.3.2 Training with Input Validation ... 10
4.3.3 Metrics and Feature Importance ... 10
4.4 Application Interface Implementation ... 11
4.4.1 Sidebar and Data Selection ... 11
4.4.2 Single-Passenger Prediction ... 11
4.4.3 Batch Prediction for Kaggle's Test File ... 11
4.4.4 Model Results and Guide Tabs ... 11
5 Results and Discussion **12**
5.1 Functional Features Achieved ... 12
5.2 System Performance ... 12
5.3 Application Interface ... 13
6 Conclusion and Future Scope **15**
6.1 Conclusion ... 15
6.2 Future Scope ... 15
Bibliography **16**

Abstract

This report presents the design and development of an interactive, web-based machine-learning application, the *Titanic Survival Explorer*, built with Python, Streamlit, pandas and scikit-learn. The application trains a Random Forest classifier on Titanic passenger records, evaluates it on a stratified hold-out set of passengers that were not used for evaluation-time training, and lets the user explore the results through accuracy, precision, recall, F1 score, a confusion matrix and feature-importance scores. Users can estimate the survival probability of a single hypothetical passenger through a form, or upload Kaggle's `test.csv` to generate a downloadable prediction file in the Kaggle submission format. The system follows a layered architecture in which a Streamlit interface communicates with a data-preparation layer and a scikit-learn pipeline that combines imputation, one-hot encoding and the classifier. Input validation, result caching and a fixed random seed make the application robust and repeatable. The project demonstrates the practical application of modern data-science tools to an end-to-end classification problem and serves as a foundation for future enhancements such as model comparison, hyper-parameter search and model explainability.

Chapter 1

Introduction

1.1 Background

Machine learning has become a core part of modern software: models learn patterns from historical records and use them to make predictions about new cases in healthcare, finance, retail and transport. A model that lives only inside a notebook, however, is hard to explore and share with people who do not program.

The sinking of the RMS Titanic in 1912 is one of the best-known teaching datasets in data science. The Kaggle "Titanic: Machine Learning from Disaster" competition provides a labelled `train.csv` and an unlabelled `test.csv` whose outcomes must be predicted. The task is a small binary classification problem that still demonstrates cleaning, feature engineering, evaluation and prediction.

This project wraps the complete workflow in a single Streamlit web application, using pandas, NumPy and scikit-learn for data preparation and modelling and Streamlit for the interactive interface.

1.2 Problem Statement

There is a clear need for a well-structured, interactive application that:

- Trains a classification model directly from an uploaded or bundled CSV file.
- Evaluates the model honestly on passengers that were held back from training.
- Predicts the survival probability of a single passenger entered by the user.
- Generates predictions for Kaggle's `test.csv` as a downloadable submission file.
- Validates its inputs and reports clear error messages.

1.3 Objectives

The primary objectives of this project are:

1. To build a fully functional, interactive machine-learning web application using Python and Streamlit.
2. To implement a reproducible scikit-learn pipeline with feature engineering.
3. To evaluate on a stratified hold-out set and present accuracy, precision, recall, F1 score, a confusion matrix and feature importances.
4. To allow users to tune the number of trees and the tree depth of the Random Forest and immediately see the effect.
5. To support both single-passenger prediction and batch prediction for Kaggle's test file with downloadable output.

1.4 Scope of the Project

The scope of this project covers the complete development of a web-based classification application. It includes data validation, feature engineering, model training and evaluation, the interactive interface, and single and batch prediction. It is intended for educational use only.

Chapter 2

Technologies Used

2.1 Frontend Technologies

2.1.1 Streamlit

Streamlit is an open-source Python framework for turning scripts into interactive web applications. The user interface is written entirely in Python: each call such as `st.title` or `st.metric` adds an element to the page, and the script is re-run from top to bottom whenever the user interacts with a widget. In this project Streamlit renders the whole interface, so no separate HTML, CSS or JavaScript code is required.

2.1.2 Streamlit Layout and Input Widgets

The application uses the sidebar for data selection and model settings, `st.tabs` to organise four views (model results, single prediction, batch prediction and a guide), `st.columns` for metric cards and form fields, and `st.form` to group the passenger inputs so that a prediction is made only when the user submits. Sliders, select boxes and number inputs constrain the values the user can enter.

2.1.3 Caching with st.cache_resource

Training a Random Forest on every page interaction would be wasteful. The `train_model` function is decorated with `st.cache_resource`, so the trained model and its evaluation results are stored in memory and reused until the training file or the model settings change.

2.1.4 File Upload and Download

`st.file_uploader` lets the user supply `train.csv` and `test.csv`, `st.download_button` delivers the generated predictions as a CSV file, and `st.link_button` points the user to the Kaggle data page.

2.2 Data and Machine-Learning Technologies

2.2.1 Python

Python 3 is the implementation language. The standard library supplies `pathlib` for locating the default CSV files next to `app.py` and `io.BytesIO` for reading uploaded bytes as a file. Type hints and `from __future__ import annotations` are used to keep function signatures clear.

2.2.2 pandas

pandas provides the `DataFrame` structure used to read CSV files, clean columns, extract the passenger title from the name, compute family-size features and assemble the confusion matrix, feature-importance table and submission file.

2.2.3 NumPy

NumPy supplies the array types underlying pandas and scikit-learn, and the confusion matrix returned by scikit-learn is a NumPy array.

2.2.4 scikit-learn

scikit-learn supplies the modelling tools: `SimpleImputer` fills missing values, `OneHotEncoder` converts text categories into numeric columns, `ColumnTransformer` applies different preprocessing to numeric and categorical columns, `Pipeline` chains preprocessing and the classifier into one object, `train_test_split` creates the stratified hold-out set, and the `metrics` module computes accuracy, precision, recall, F1 score and the confusion matrix.

2.2.5 Random Forest Classifier

A Random Forest is an ensemble of decision trees, each trained on a random sample of the data and a random subset of features; the final prediction is obtained by combining the votes of all trees. It handles mixed feature types, captures non-linear relationships and is relatively resistant to over-fitting. The forest also yields feature-importance scores, which the application displays.

2.3 Dataset

2.3.1 Kaggle Titanic Dataset

The dataset comes from the Kaggle Titanic competition. The training file contains the columns `Survived`, `Pclass`, `Name`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare` and `Embarked`, among others, while the test file has the same columns without `Survived`. The application places the CSV files in the project directory or accepts them through the upload widgets.

2.4 Infrastructure

2.4.1 Runtime Environment

The application is a single Python process started with the Streamlit command-line tool. It holds data and models in memory, needs no database, and can be run on a local computer or on any server or cloud platform that supports Python and Streamlit. Users access it through a web browser.

Table 2.1: Summary of Technologies Used

Layer	Technology	Purpose
Interface	Streamlit	Web UI, widgets, tabs, caching
Language	Python 3	Application logic
Data handling	pandas, NumPy	CSV reading, cleaning, tables
Preprocessing	SimpleImputer, OneHotEncoder, ColumnTransformer	Missing values and categories
Modelling	scikit-learn, RandomForestClassifier	Training and prediction
Evaluation	sklearn.metrics, train_test_split	Hold-out validation and metrics
Data source	Kaggle Titanic CSV files	Training and test records
Runtime	Streamlit server	Hosting of the application

Chapter 3

System Architecture

3.1 Overview

The application follows a three-layer architecture. The *presentation layer* is the Streamlit interface (sidebar, tabs and widgets). The *application-logic layer* consists of the Python functions that read and validate CSV files, engineer features, build, train and evaluate the model, and produce predictions. The *data layer* consists of the Kaggle CSV files, which are either bundled in the project directory or uploaded by the user, and an in-memory cache holding the trained model.

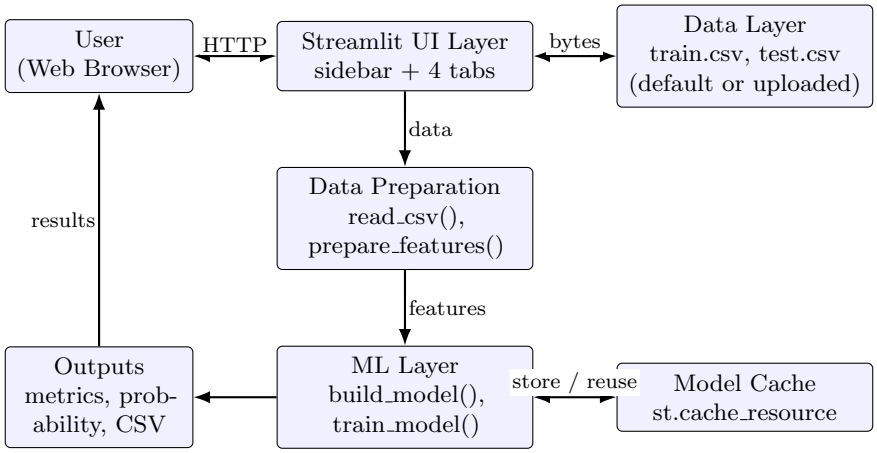


Figure 3.1: High-Level System Architecture

3.2 Application Module Architecture

All code resides in one module, `app.py`. The entry function `main()` builds the sidebar, obtains the training bytes (uploaded file first, then the bundled `train.csv`), calls the cached `train_model()` function and then fills four tabs. If no training data is available, or training fails validation, the application shows an explanatory message and stops cleanly.

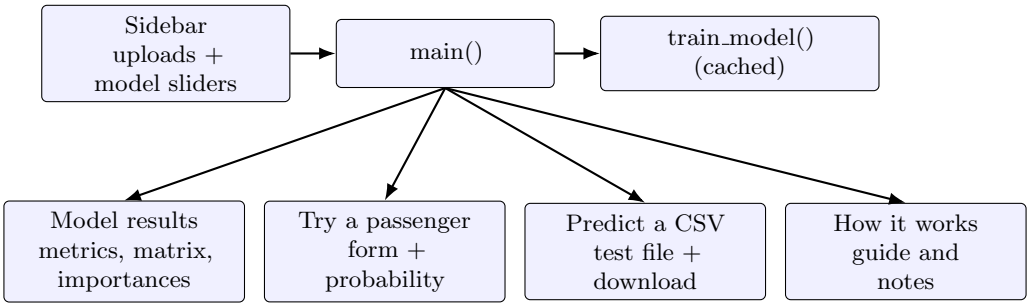


Figure 3.2: Application Module and Tab Structure

3.3 Model Training and Evaluation Architecture

Training is performed in two steps. First, the validated data are split with a stratified 80/20 split, a validation model is fitted on the 80% part, and its predictions on the unseen 20%

produce the reported metrics. Second, a final model with identical settings is fitted on *all* labelled rows and is the model used for every prediction in the application. The results, including the confusion matrix and feature importances, are cached together.

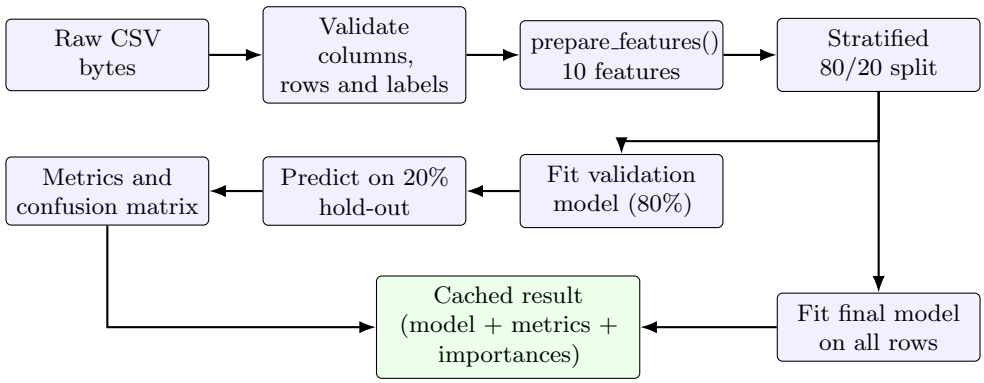


Figure 3.3: Model Training and Evaluation Flow

3.4 Feature and Data Schema Design

The model never sees passenger identifiers or ticket numbers. From the eight raw columns it uses, the application builds ten features, listed in Table 3.1. The raw name is used only to extract a title.

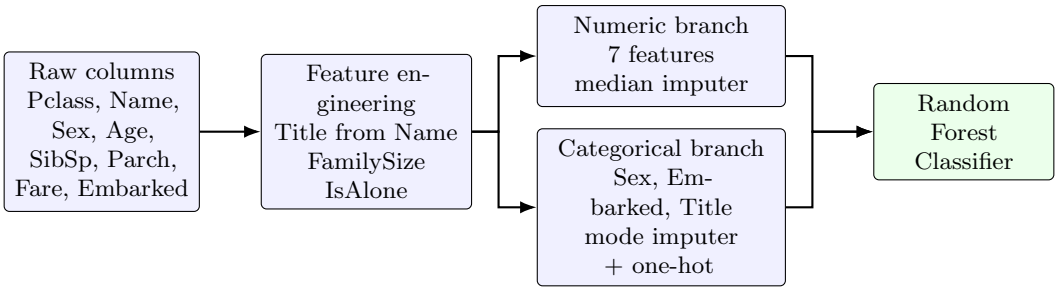


Figure 3.4: Feature Preparation and Pipeline Schema

Table 3.1: Features Used by the Model

Feature	**Type**	**Description**
Pclass	Numeric	Ticket class (1, 2 or 3)
Age, Fare	Numeric	Age in years; ticket fare
SibSp, Parch	Numeric	Siblings/spouses and parents/children aboard
FamilySize	Numeric (derived)	SibSp + Parch + 1
IsAlone	Numeric (derived)	1 if FamilySize equals 1, else 0
Sex, Embarked	Categorical	Sex and port of embarkation (S, C, Q)
Title	Categorical (derived)	Mr, Mrs, Miss, Master or Other, taken from Name

3.5 Prediction Flow

When the user submits the passenger form, the application converts the inputs into a one-row DataFrame, passes it through the same `prepare_features()` function used for training and asks the cached pipeline for the survival probability. Because the preprocessing is part of the pipeline, the user's values are imputed and encoded exactly like the training data.

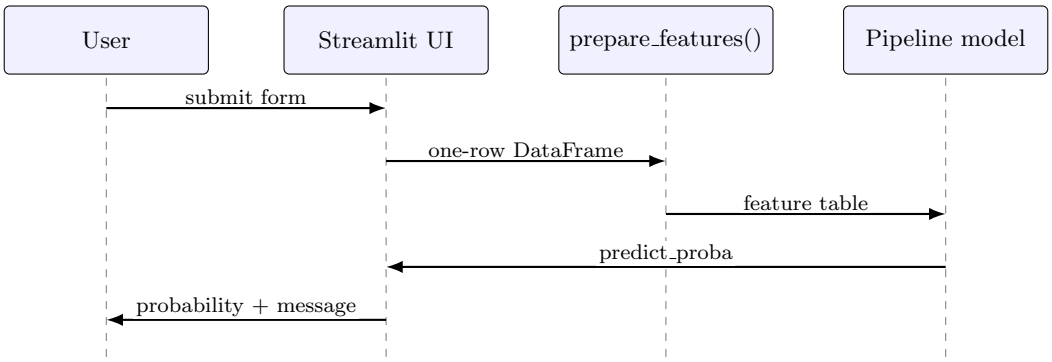


Figure 3.5: Single-Passenger Prediction Flow

Chapter 4

Implementation Details

4.1 Project Structure

The project is deliberately compact: a single source file, `app.py`, plus the two Kaggle data files, `train.csv` and `test.csv`, placed in the same directory. The directory is located at run time with `Path(__file__).resolve().parent`, so the application works from any location. The code inside `app.py` is divided into small functions: `read_csv`, `prepare_features`, `build_model`, `train_model` and `main`.

4.2 Data Handling Implementation

4.2.1 Configuration and Constants

The module defines the default file locations, the set of required training columns and the lists of numeric and categorical features. Keeping these in constants guarantees that training and prediction always use the same columns in the same order.

```
PROJECT_DIR = Path(__file__).resolve().parent
DEFAULT_TRAIN_FILE = PROJECT_DIR / "train.csv"
DEFAULT_TEST_FILE = PROJECT_DIR / "test.csv"

NUMERIC_FEATURES = ["Pclass", "Age", "SibSp", "Parch", "Fare", "FamilySize", "IsAlone"]
CATEGORICAL_FEATURES = ["Sex", "Embarked", "Title"]
FEATURE_NAMES = NUMERIC_FEATURES + CATEGORICAL_FEATURES
```

Listing 4.1: Configuration constants (app.py)

4.2.2 Reading CSV Files

Both uploaded and bundled files are handled as raw bytes, which also makes them suitable as cache keys. The `read_csv` function converts parsing and encoding problems into a readable error message and strips stray spaces from column names (`pd.read_csv(BytesIO(file_bytes))` wrapped in a `try` block that raises a `ValueError`).

4.2.3 Feature Engineering

The `prepare_features` function first checks that all eight required passenger columns exist. It then extracts the title (the word between the comma and the first full stop in the name), merges the variants Mlle, Ms and Mme into Miss and Mrs, and groups every title other than Mr, Mrs, Miss and Master into "Other". Family size and a lone-traveller flag are derived from `SibSp` and `Parch`. Finally numeric columns are coerced to numbers and text columns are cleaned, with blanks replaced by "Unknown".

```
features["Title"] = (
    features["Name"].fillna("").astype(str)
    .str.extract(r",\s*([^\.]+)\.", expand=False)
    .fillna("Unknown").str.strip()
)
features["Title"] = features["Title"].replace({"Mlle": "Miss", "Ms": "Miss", "Mme": "Mrs"})
common_titles = {"Mr", "Mrs", "Miss", "Master"}
features["Title"] = features["Title"].where(features["Title"].isin(common_titles), "Other")
features["FamilySize"] = (
    pd.to_numeric(features["SibSp"], errors="coerce").fillna(0)
    + pd.to_numeric(features["Parch"], errors="coerce").fillna(0) + 1
```

```
)
features["IsAlone"] = (features["FamilySize"] == 1).astype(int)
```

Listing 4.2: Title and family features (app.py)

4.3 Model Implementation

4.3.1 Preprocessing and Classifier Pipeline

The `build_model` function returns a complete scikit-learn pipeline. Numeric columns receive median imputation; categorical columns receive most-frequent imputation followed by one-hot encoding, with `handle_unknown="ignore"` so that categories not seen in training do not cause errors. The classifier is a Random Forest whose number of trees and depth come from the sidebar sliders; `min_samples_leaf=2`, balanced class weights and a fixed `random_state` of 42 make it stable and repeatable.

```
def build_model(number_of_trees: int, maximum_depth: int) -> Pipeline:
    numeric_steps = Pipeline(steps=[("fill_missing", SimpleImputer(strategy="median"))])
    categorical_steps = Pipeline(steps=[
        ("fill_missing", SimpleImputer(strategy="most_frequent")),
        ("encode_categories", OneHotEncoder(handle_unknown="ignore")),
    ])
    preprocessing = ColumnTransformer(transformers=[
        ("numbers", numeric_steps, NUMERIC_FEATURES),
        ("categories", categorical_steps, CATEGORICAL_FEATURES),
    ])
    classifier = RandomForestClassifier(
        n_estimators=number_of_trees, max_depth=maximum_depth,
        min_samples_leaf=2, class_weight="balanced", random_state=42, n_jobs=-1,
    )
    return Pipeline(steps=[("preprocess", preprocessing), ("classifier", classifier)])
```

Listing 4.3: Pipeline construction (app.py)

4.3.2 Training with Input Validation

`train_model` is cached and begins by validating the data: all required columns must exist, at least ten rows must be present, `Survived` must contain only 0 or 1, and each outcome must occur at least twice (which a stratified split requires). Any violation raises a `ValueError` that the interface turns into a friendly error message. The features are then built with `prepare_features` and split with `train_test_split(test_size=0.2, random_state=42, stratify=labels)`, so both parts keep the original survival ratio.

4.3.3 Metrics and Feature Importance

The validation model is fitted on the training part and evaluated on the hold-out part. A second, final model is then fitted on every labelled row. Its feature importances are read from the forest, the prefixes added by the column transformer are removed, importances of one-hot columns are summed per feature name, and the twelve largest are kept for display.

```
validation_model = build_model(number_of_trees, maximum_depth)
validation_model.fit(train_x, train_y)
predictions = validation_model.predict(valid_x)
...
return {
    "model": final_model, "row_count": len(training_data),
    "survival_rate": float(labels.mean()),
    "metrics": {
        "accuracy": float(accuracy_score(valid_y, predictions)),
        "precision": float(precision_score(valid_y, predictions, zero_division=0)),
        "recall": float(recall_score(valid_y, predictions, zero_division=0)),
        "f1": float(f1_score(valid_y, predictions, zero_division=0)),
    },
    "confusion_matrix": confusion_matrix(valid_y, predictions, labels=[0, 1]),
```

```
    "feature_importance": feature_importance, "validation_count": len(valid_y),
}
```

Listing 4.4: Evaluation metrics and result dictionary (app.py)

4.4 Application Interface Implementation

4.4.1 Sidebar and Data Selection

The sidebar offers two uploaders and two sliders. The number of trees ranges from 50 to 500 in steps of 50 (default 200) and the tree depth from 2 to 16 (default 6). An uploaded training file takes priority; otherwise the bundled `train.csv` is used. If neither exists, the application displays instructions and a link to the Kaggle data page and stops.

4.4.2 Single-Passenger Prediction

The "Try a passenger" tab contains a form with name, class, sex, age, family counts, fare and port of embarkation. On submission the values are formatted as a DataFrame and passed through the cached model. A probability of 0.5 or above is reported as "more likely to survive"; the tab also reminds the user that the result is a learned pattern and not a certainty.

```
if submitted:
    passenger = pd.DataFrame([{"Name": name, "Pclass": passenger_class, "Sex": sex,
        "Age": age, "SibSp": sib_sp, "Parch": parch, "Fare": fare, "Embarked": embarked}])
    probability = float(result["model"].predict_proba(prepare_features(passenger))[0, 1])
    st.metric("Estimated survival probability", f"{probability:.1%}")
    if probability >= 0.5:
        st.success("The model predicts: **more likely to survive**.")
    else:
        st.info("The model predicts: **more likely not to survive**.")
```

Listing 4.5: Probability estimate for one passenger (app.py)

4.4.3 Batch Prediction for Kaggle's Test File

The "Predict a CSV" tab uses an uploaded `test.csv` or the bundled one. It prepares the features, predicts the outcome of every passenger, attaches the `PassengerId` column (or a generated sequence if it is absent) and shows the first twenty rows. The full table can be downloaded as `titanic_predictions.csv`, the format required by Kaggle.

```
test_data = read_csv(test_bytes, "test")
test_features = prepare_features(test_data)
test_predictions = result["model"].predict(test_features).astype(int)
submission = pd.DataFrame({"PassengerId": passenger_ids, "Survived": test_predictions})
st.dataframe(submission.head(20), width="stretch", hide_index=True)
st.download_button("Download Kaggle predictions CSV",
    submission.to_csv(index=False).encode("utf-8"),
    file_name="titanic_predictions.csv", mime="text/csv")
```

Listing 4.6: Batch prediction and download (app.py)

4.4.4 Model Results and Guide Tabs

The "Model results" tab shows the four metrics as cards with plain-language definitions, a labelled confusion matrix and the feature-importance table, each with a caption stating that the numbers are estimates and that importance does not prove causation. The "How it works" tab explains the five steps of the application, states that identifiers and ticket numbers are not used, and warns that the model can reflect historical bias.

Chapter 5

Results and Discussion

5.1 Functional Features Achieved

The following features were successfully implemented and verified during testing of the application:

- **Flexible Data Input:** The training CSV and an optional test CSV can be uploaded in the sidebar; if they are not uploaded, `train.csv` and `test.csv` from the project folder are used automatically.
- **Input Validation:** Missing columns, too few rows, labels other than 0 and 1, and unreadable files are detected and reported through clear on-screen messages instead of crashes.
- **Feature Engineering:** A passenger title is extracted from the name, and family size and a lone-traveller flag are derived, giving ten features per passenger.
- **Robust Preprocessing:** Missing numeric values are filled with the median and missing categories with the most frequent value; unseen categories are ignored safely during encoding.
- **Honest Evaluation:** A stratified 20% hold-out set is used for accuracy, precision, recall and F1 score, and a confusion matrix shows the types of errors made.
- **Adjustable Model:** The number of trees (50–500) and the tree depth (2–16) can be changed, and the model is re-trained with the new settings.
- **Feature Importance:** The twelve most influential features are listed with their relative importance scores.
- **Single-Passenger Prediction:** The user enters passenger details and receives a survival probability together with a plain-language prediction.
- **Batch Prediction and Download:** Predictions for every passenger in `test.csv` are generated and downloaded in the Kaggle submission format.
- **Reproducibility and Caching:** A fixed random seed makes results repeatable, and caching avoids re-training when only unrelated widgets change.

5.2 System Performance

The application is lightweight and performs reliably under normal usage. The Kaggle training file is small (891 passenger records, of which 38.4% survived), so a Random Forest with the default 200 trees of depth 6 trains quickly, and the stratified 20% hold-out then contains 179 passengers. Training uses all available processor cores through `n_jobs=-1`, and the result is cached by `st.cache_resource`, so switching between tabs or entering passenger details does not trigger re-training; only a change of the training file or of the two model sliders does. Single-passenger predictions require only one pass of a one-row table through the fitted pipeline and therefore respond almost instantly.

With the default settings on the Kaggle training file, the application reported an accuracy of 82.7%, precision of 77.1%, recall of 78.3% and an F1 score of 77.7% on the 179 hold-out passengers (Figure 5.1). The confusion matrix shows 94 correct "did not survive" predictions, 16 false "survive" predictions, 15 missed survivors and 54 correct "survive" predictions. The most influential features were `Title_Mr` (0.176) and `Sex_male` (0.156). For the example passenger

in Figure 5.2 the model estimated a survival probability of 44.4%, and for Kaggle's `test.csv` it generated predictions for all 418 passengers (Figure 5.3).

The metrics are computed live from the supplied data and settings, so they are not hard-coded; they should be read as estimates, because a single random hold-out split of a small dataset can vary. The fixed random seed ensures that the same data and settings always give the same figures, which makes experiments with the sliders easy to compare.

5.3 Application Interface

Figures 5.1 to 5.3 show screenshots of the running application (default settings of 200 trees and depth 6). The sidebar on the left holds the data and model settings, while the main area holds the four tabs.

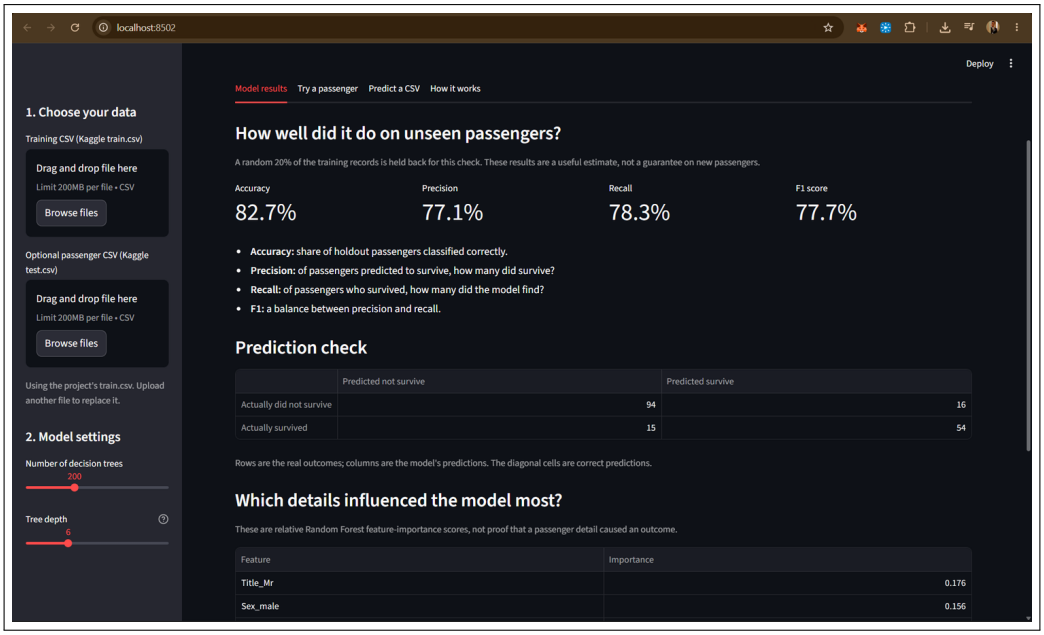


Figure 5.1: Model Results Tab: Metrics, Confusion Matrix and Feature Importance

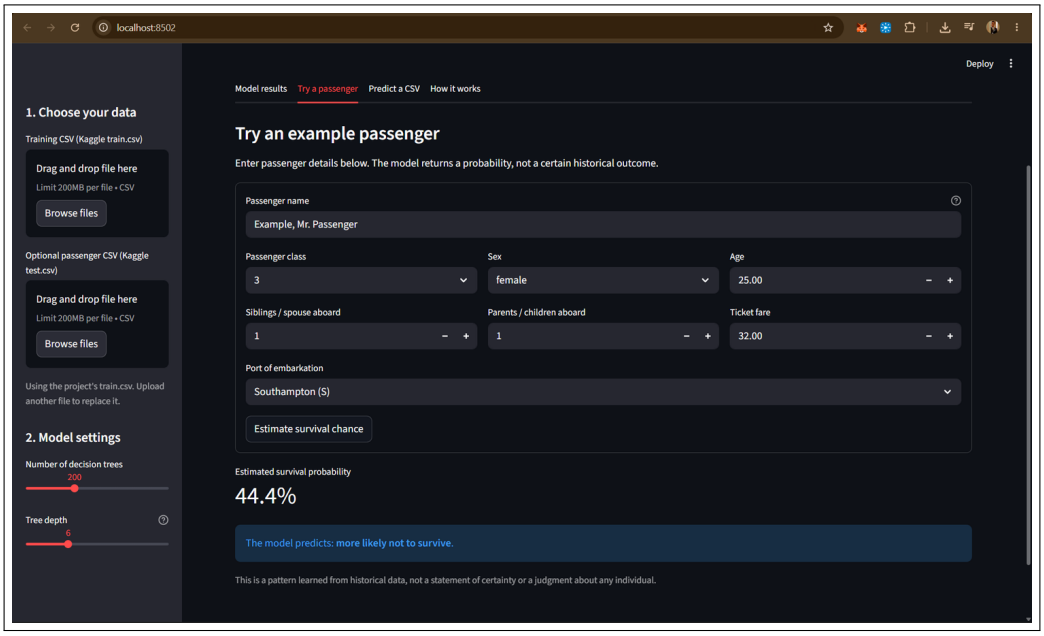


Figure 5.2: Try a Passenger Tab: Single-Passenger Survival Probability

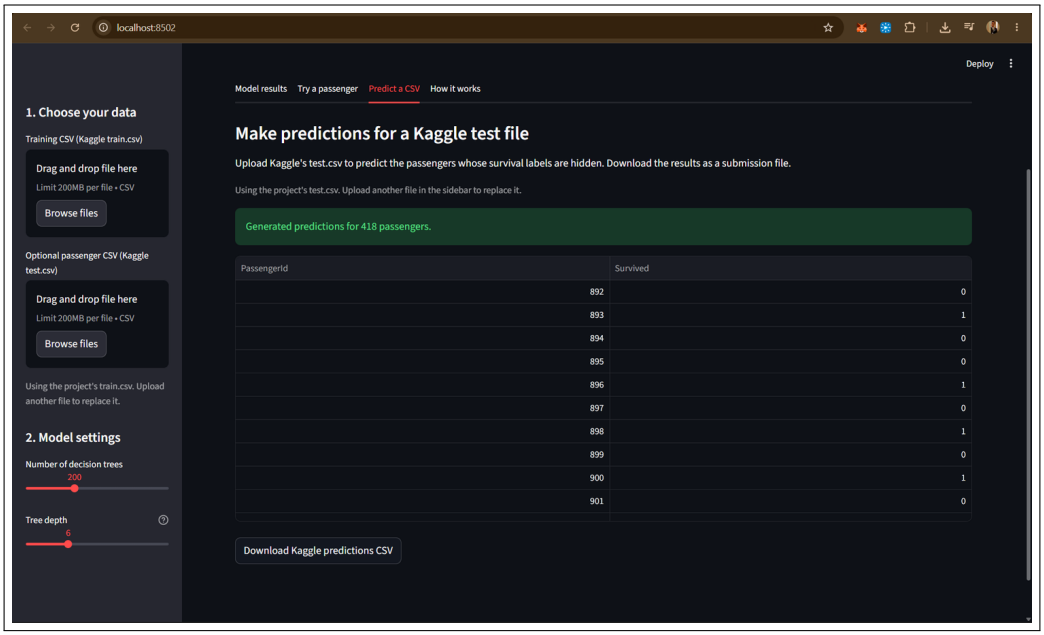


Figure 5.3: Predict a CSV Tab: Batch Predictions and Download Button

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

This project demonstrates the development of a complete, interactive machine-learning application using Python, Streamlit, pandas and scikit-learn. Starting from raw passenger records, every stage of the workflow was implemented: validation, cleaning, feature engineering, pipeline construction, evaluation on unseen data, interactive prediction and downloadable batch output.

The combination of a scikit-learn pipeline, which keeps preprocessing and the classifier together, and a Streamlit interface, which re-runs on every interaction while caching the expensive training step, proved to be an effective and practical architecture. Evaluating on a stratified hold-out set before fitting the final model on all rows gives an honest performance estimate without sacrificing training data for the model that is actually used.

The application satisfies all of the objectives set out at the beginning of the project. It is functional, reproducible and defensive against bad input, and it states clearly that the model reflects historical patterns and must not be used for decisions about real people. It provides a solid understanding of how a data-science workflow is turned into a usable application, and it serves as a strong base for further development.

6.2 Future Scope

Several meaningful enhancements can be made in future development cycles:

- **Model Comparison:** Adding other classifiers such as logistic regression, gradient boosting or support vector machines, and comparing them side by side.
- **Cross-Validation:** Replacing the single hold-out split with k-fold cross-validation to give more stable performance estimates.
- **Hyper-parameter Search:** Offering automated tuning with grid or randomised search instead of manual sliders.
- **Model Explainability:** Integrating tools such as SHAP or permutation importance to explain individual predictions.
- **Richer Visualisation:** Adding charts for survival rate by class, sex and age, and ROC or precision-recall curves.
- **Model Export:** Allowing users to download the trained pipeline for reuse in other programs.
- **Additional Features:** Using cabin and ticket information with careful treatment of missing values to improve accuracy.
- **Other Datasets:** Generalising the interface so that any labelled CSV file can be used for binary classification.
- **Cloud Deployment:** Hosting the application on a public cloud platform with authentication and usage logging.

Bibliography

[1] Snowflake Inc., *Streamlit Documentation*, Available at: `https://docs.streamlit.io/`, Accessed: October 2026.

[2] scikit-learn developers, *scikit-learn: Machine Learning in Python – User Guide*, Available at: `https://scikit-learn.org/stable/`, Accessed: October 2026.

[3] The pandas development team, *pandas Documentation*, Available at: `https://pandas.pydata.org/docs/`, Accessed: October 2026.

[4] NumPy developers, *NumPy Documentation*, Available at: `https://numpy.org/doc/`, Accessed: October 2026.

[5] Python Software Foundation, *Python 3 Documentation*, Available at: `https://docs.python.org/3/`, Accessed: October 2026.

[6] Kaggle, *Titanic – Machine Learning from Disaster*, Available at: `https://www.kaggle.com/competitions/titanic`, Accessed: October 2026.

[7] L. Breiman, *Random Forests*, Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[8] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] W. McKinney, *Data Structures for Statistical Computing in Python*, Proceedings of the 9th Python in Science Conference, pp. 56–61, 2010.

[10] C. R. Harris et al., *Array programming with NumPy*, Nature, vol. 585, pp. 357–362, 2020.

[11] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, New York, USA, 2009.

[12] M. Sokolova and G. Lapalme, *A systematic analysis of performance measures for classification tasks*, Information Processing & Management, vol. 45, no. 4, pp. 427–437, 2009.

[13] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O'Reilly Media, Sebastopol, CA, USA, 2022.